
CSCI 2590: Assignment 4

zz5070

NetID: zz5070

Q0. 1.

GitHub repository:

<https://github.com/Joni-z/CSCI-GA-2590-Natural-Lang-Processing/tree/main/HW4>

Q2. 1.

My out-of-distribution transformation simulates informal mobile typing noise while preserving the sentiment label of the movie review. For each input review, I lowercase the text, tokenize it with NLTK's word tokenizer, and consider only alphabetic tokens longer than three characters. I seed a local random number generator using an MD5 hash of the original text so that the transformation is deterministic for a given example.

For each eligible token, I apply one of four character-level perturbations with total probability 45%: with probability 22%, I replace one character by a neighboring key on a QWERTY keyboard; with probability 9%, I delete one vowel; with probability 8%, I duplicate one alphabetic character; and with probability 6%, I swap two adjacent internal alphabetic characters. Punctuation, numbers, and short words are left unchanged, and the transformed tokens are detokenized back into a sentence using `TreebankWordDetokenizer`.

This is a reasonable transformation because user-generated text commonly contains small typos, missing letters, or repeated characters, especially in informal reviews. These changes usually do not alter the semantic content or sentiment label. For example, a positive review remains positive even if a word like "excellent" is typed as "excellnt" or "excellnt".

Q3. 1

Report & Analysis

- The BERT model trained on the original IMDB training set achieved 93.116% accuracy on the original test data and 86.924% accuracy on the transformed test data.
- The model trained with augmented data achieved 93.080% accuracy on the original test data and 89.432% accuracy on the transformed test data.
- Data augmentation improved performance on the transformed test set from 86.924% to 89.432%, a gain of 2.508 percentage points. This suggests that exposing the model to typo-like perturbations during training made it more robust to the same kind of distribution shift at evaluation time.
- Data augmentation very slightly reduced performance on the original test set from 93.116% to 93.080%, a decrease of 0.036 percentage points. This small tradeoff is expected: the augmented training distribution is noisier than the original one, so the model spends some capacity becoming invariant to spelling perturbations rather than fitting only clean text.
- Intuitively, augmentation teaches the classifier that small spelling changes should not change the label. This reduces the model's reliance on exact surface forms and encourages more robust lexical representations. However, because the transformation only covers a narrow family of character-level noise, it does not protect against

other OOD shifts such as paraphrases, domain changes, sarcasm, or more severe grammar errors.

Part II. Q4

Statistics Name	Train	Dev
Number of examples	4225	466
Mean sentence length	10.96	10.91
Mean SQL query length	60.90	58.90
Vocabulary size (natural language)	868	444
Vocabulary size (SQL)	643	393

Table 1: Data statistics before preprocessing, computed using whitespace tokenization over the raw .nl and .sql files.

T5 fine-tuned model	Train	Dev
Mean encoder token length	23.10	23.07
Max encoder token length	65	49
Mean decoder token length	211.14	205.51
Max decoder token length	384	384
Active encoder subword vocabulary	796	470
Active decoder subword vocabulary	555	395

Table 2: Data statistics after preprocessing with the `t5-small` tokenizer. Encoder inputs use the prefix `translate English to SQL:`. Encoder sequences are truncated to 128 tokens and decoder SQL sequences are truncated to 384 tokens.

Q5

Design choice	Description
Data processing	I converted each natural-language input into a text-to-text prompt of the form <code>translate English to SQL: query</code> . SQL targets were used as decoder targets for train and dev splits. For test examples, only the encoder input and the initial decoder token are provided. Batches use dynamic padding.
Tokenization	I used HuggingFace’s <code>T5TokenizerFast</code> from <code>t5-small</code> for both encoder inputs and decoder SQL outputs. Encoder inputs were tokenized with maximum length 128 and SQL outputs with maximum length 384. The decoder input is the target SQL shifted right by one position, with the T5 pad token used as the decoder start token.
Architecture	I fine-tuned the full pretrained <code>t5-small</code> encoder-decoder model. I did not freeze any layers; all parameters were updated during fine-tuning. For final generation, I used beam search with 4 beams and maximum 512 generated tokens.
Hyperparameters	The fine-tuned model used AdamW, learning rate 3×10^{-4} , weight decay 0, batch size 16, dev/test batch size 16 during training, a linear learning-rate scheduler, 1 warmup epoch, maximum 10 epochs, and patience 3. The best checkpoint was selected using development Record F1. Final test inference used batch size 16 or 32 depending on available hardware, with the same beam-search settings. I also applied a conservative SQL cleanup pass to generated queries with execution errors, fixing duplicate aliases, missing comparison operators, unmatched quotes, and small parenthesis imbalances.

Table 3: Details of the best-performing T5 model configuration.

Q6.

System	SQL Query EM	Record EM	Record F1
Dev Results			
T5 fine-tuned final submitted output	1.93	60.73	70.49
T5 fine-tuned best epoch during training	1.93	54.94	63.38
Test Results			
T5 fine-tuned	Hidden test labels unavailable locally; submitted files generated.		

Table 4: Development and test results. Dev values are percentages. The hidden test set does not include local ground-truth SQL or records, so only the submitted prediction files can be generated locally.

Quantitative Results: On the development set, the final submitted output achieved Record F1 70.49, Record EM 60.73, and SQL Query EM 1.93. During training, the strongest logged pre-cleanup Record F1 was 63.38 at epoch 7. Increasing generation length to 512 tokens and applying the conservative SQL cleanup pass reduced dev execution errors from 119 to 38. The final test SQL and record files contain 432 predictions/records, matching the 432 test natural-language inputs. Executing generated test SQL locally produced 34 execution errors out of 432 queries, an execution-error rate of 7.87%; this is not a test-set accuracy metric, but it is a useful sanity check on output validity.

Error Type	Example Of Error	Error Description	Statistics
SQL syntax error	arrival.time flight_1...	Some generated queries still have malformed conditions or parentheses after cleanup, preventing SQLite from parsing the query.	12/466 dev queries
Execution time-out or interruption	flights and prices from LA to Charlotte	A few generated queries are syntactically acceptable enough to start execution but produce inefficient joins or overly broad conditions, causing execution to be interrupted.	4/466 dev queries
Invalid schema reference	no such column: flight	The model sometimes hallucinates column names or references aliases/tables that are not present in the generated FROM clause. This indicates imperfect schema grounding.	22/466 dev queries

Table 5: Qualitative error analysis for the fine-tuned T5 model on the development set.

Qualitative Error Analysis: The main pattern is that the model often captures the broad intent of a query, such as cities, airlines, or time constraints, but struggles to consistently produce executable SQL for long examples. SQL Query EM is therefore much lower than Record F1: exact string matching penalizes harmless formatting or equivalent query differences, while Record F1 gives partial credit when the generated query returns similar database records.

Q7.

Google Drive checkpoint link:

<https://drive.google.com/file/d/1yVdm1LFkcTIko0TefTz-LFA1TdbCk5K7/view?usp=sharing>

The checkpoint used for Q7 is stored locally at `part-2/checkpoints/ft_experiments/experiment/best`. The submitted SQL and record files were generated from this checkpoint.